

Riassunti LAB Basi di Dati

Angelo Vaccaro

1 PostgreSQL: Architettura di Sistema e Memoria

Prima di addentrarci nei comandi SQL, è fondamentale comprendere la natura ingegneristica di PostgreSQL. Esso non è un semplice "lettore di file", ma un RDBMS (Relational Database Management System) avanzato, basato su una solida architettura **Client-Server**.

Sotto il cofano, il database non è mai accessibile direttamente. Tutte le richieste devono passare attraverso il demone **postmaster**, il processo server principale in ascolto permanente. Quando un client (come **psql** da riga di comando o l'interfaccia grafica **pgAdmin**) invia una richiesta di connessione TCP/IP, il *postmaster* valuta le credenziali, alloca un blocco di memoria RAM dedicata e crea un processo figlio (*backend process*) esclusivo per gestire quella specifica sessione utente.

1.1 Tipizzazione e Strutture di Memoria (Data Types)

Quando si definisce lo schema di una tabella con il comando **CREATE TABLE**, non si sta solo dando un nome alle colonne, ma si sta impartendo un ordine preciso al motore di archiviazione su come allocare i byte sul disco fisso per ogni singola riga (tupla). L'uso corretto dei domini (Tipi di Dato) è il primo passo per l'ottimizzazione fisica del database.

1.1.1 Stringhe di Testo e Strutture Varlena

PostgreSQL gestisce i dati testuali in modo molto sofisticato, non come semplici array in linguaggio C. Tutti i tipi carattere (**CHAR**, **VARCHAR**, **TEXT**) sono memorizzati internamente utilizzando una struttura dati chiamata **varlena** (Variable Length Array).

Questa struttura impone che ogni stringa salvata su disco debba essere preceduta da un **header invisibile** (generalmente 1 o 4 byte) che indica al motore esattamente quanti byte occupa il dato puro (payload). Poiché il server utilizza la codifica **UTF-8**, un singolo carattere testuale può occupare dinamicamente da 1 a 4 byte a seconda della sua natura (es. le lettere accentate occupano più spazio delle normali lettere ASCII).

- **CHAR(n)**: Pur essendo definito "a lunghezza fissa", è anch'esso una struttura *varlena*. Se l'utente inserisce una stringa più corta di n , il motore esegue un'operazione di **padding**, riempiendo fisicamente lo spazio mancante con caratteri vuoti (spazi) fino al raggiungimento della soglia n .
- **VARCHAR(n)**: Definisce un limite massimo di capienza logica. Il motore alloca dinamicamente sul disco solo i byte strettamente necessari per il payload inserito (più l'header), senza effettuare alcuno spreco di padding spaziale.
- **TEXT**: L'architettura alloca lo spazio dinamicamente come in **VARCHAR**, ma rimuove il vincolo del limite massimo. Se il testo diventa eccessivamente voluminoso (es. un intero articolo), il motore utilizza un meccanismo interno chiamato **TOAST** (The Oversized-Attribute Storage Technique) per spezzettare e comprimere la stringa in pagine di memoria separate, lasciando nella tabella principale solo un puntatore per non compromettere le prestazioni di scansione.

1.1.2 Gestione dei Numeri: Fissi vs Approssimati

La scelta del tipo di dato numerico determina l'hardware coinvolto nei calcoli della CPU.

- **Interi Nativi**: Tipi come **SMALLINT** (16 bit / 2 byte) e **INTEGER** (32 bit / 4 byte) vengono elaborati direttamente dalla ALU (Arithmetic Logic Unit) del processore, garantendo velocità massime per chiavi primarie o conteggi.
- **Virgola Mobile (Floating-Point)**: **REAL** e **DOUBLE PRECISION** seguono gli standard hardware IEEE 754. Essi scindono il numero in *mantissa* ed *esponente*. Essendo basati sull'architettura a base 2 del processore, questi tipi non riescono a rappresentare esattamente frazioni decimali come 0.1. Causano errori di arrotondamento e per questo motivo l'ottimizzatore sconsiglia tassativamente il loro uso per applicazioni finanziarie.
- **Numerici Esatti (Arbitrary Precision)**: I tipi **NUMERIC(precisione, scala)** o **DECIMAL** risolvono il problema della perdita di precisione. Sotto il cofano, PostgreSQL non li tratta come numeri nativi per la CPU, ma come speciali stringhe matematiche elaborate via software in base 10000. L'attributo *precisione* stabilisce il numero totale massimo di cifre significative, mentre *scala* blocca rigidamente il numero di cifre decimali ammesse dopo la virgola (es. **NUMERIC(5,2)** accetta fino a 999.99).

1.2 Vincoli di Integrità Strutturale (Constraints)

Quando si applica un vincolo (es. **NOT NULL**, **UNIQUE**, **CHECK**) nella clausola **CREATE TABLE** o **ALTER TABLE**, PostgreSQL non si limita a registrarlo come regola astratta, ma interviene a basso livello.

- **CHECK:** Il motore SQL inietta un blocco di validazione. Prima che una tupla venga trasferita dalla memoria RAM (buffer) al disco rigido, il sistema valuta l'espressione logica (es. `Stipendio >= 0`). Se l'espressione restituisce `FALSE`, la transazione corrente viene abortita e il salvataggio negato.
- **UNIQUE e PRIMARY KEY:** Quando si dichiarano questi vincoli, PostgreSQL esegue un'operazione **hardware-intensive**: crea in automatico e silente un **Indice B-Tree** nascosto associato a quella colonna. Il controllo di unicità non viene fatto leggendo sequenzialmente tutta la tabella (operazione folle per tabelle di milioni di righe), ma interrogando istantaneamente l'albero B-Tree. La **PRIMARY KEY** agisce combinando logicamente il vincolo **UNIQUE** e il **NOT NULL** sullo stesso attributo.

1.3 Integrità Referenziale: La logica dei Puntatori (FOREIGN KEY)

I vincoli interrelazionali (**FOREIGN KEY** e **REFERENCES**) sono l'anima dei database relazionali. Essi creano un vincolo di dipendenza strutturale tra i record di una tabella *Slave* (interna) e i record identificanti (obbligatoriamente definiti come **PRIMARY KEY** o **UNIQUE**) di una tabella *Master* (esterna).

Sotto il cofano, PostgreSQL impone queste regole piazzando dei "trigger" invisibili su entrambe le tabelle. Quando un'applicazione esegue una **INSERT** nella tabella *Slave* (es. assegna un vigile a un'infrazione), il motore interrompe l'operazione, si reca nell'indice B-Tree della tabella *Master* (es. Vigili) e cerca la matricola. Se l'indice non la trova, blocca l'inserimento.

Questa dipendenza pone un problema architetturale critico durante le operazioni distruttive (**DELETE**) o di mutazione (**UPDATE**) sulla tabella *Master*. Il programmatore deve istruire l'ottimizzatore su come comportarsi in caso di violazione:

- **NO ACTION / RESTRICT:** È la barriera di sicurezza di default. Se si tenta di eliminare un record *Master* che possiede ancora righe *Slave* collegate (es. eliminare un dipartimento pieno di impiegati), il trigger scatta e abortisce brutalmente la transazione sollevando un'eccezione SQL.
- **CASCADE:** È il meccanismo dell'effetto domino. L'eliminazione o la modifica del record *Master* innesca una reazione a catena per cui il DBMS cancella autonomamente e definitivamente dal disco anche tutti i record *Slave* associati, mantenendo il database pulito ma rischiando distruzioni massive non calcolate.
- **SET NULL / SET DEFAULT:** Invece di distruggere le righe orfane, il motore esegue una **UPDATE** forzata sulle righe *Slave* collegate, sovrascrivendo la chiave esterna con il marker **NULL** (se consentito dallo schema) o con un valore **DEFAULT** prefissato, "sganciando" di fatto la dipendenza logica.

2 Interrogazioni SQL: Query, Pattern Matching e Raggruppamento

L'essenza di un database relazionale non risiede solo nell'archiviare i dati, ma nella capacità di estrarli, incrociarli e analizzarli in modo logico. Il Data Query Language (DQL), incarnato principalmente dall'istruzione **SELECT**, fornisce un arsenale sintattico che permette di trasformare una montagna di byte su disco in informazioni ad alto valore aggiunto.

2.1 La Struttura Anatomica della SELECT

Il comando **SELECT** è un costrutto dichiarativo puro: descriviamo la forma del risultato desiderato, e l'ottimizzatore interno (come analizzato in precedenza) si occupa del "come" reperirlo. Sotto il cofano, l'esecuzione logica di una query procede a strati successivi:

1. **FROM:** Il database individua le relazioni sorgente allocate sul disco. Se sono specificate più tabelle separate da virgole, il motore esegue (logicamente) un pesantissimo prodotto cartesiano (**CROSS JOIN**), abbinando ogni riga della prima tabella con ogni riga della seconda.
2. **WHERE:** È il primo vero filtro. Il database applica l'espressione booleana riga per riga. Le tuple che restituiscono **FALSE** o **UNKNOWN** vengono scartate.
3. **GROUP BY e HAVING:** Se presenti, le tuple sopravvissute al **WHERE** vengono aggregate in partizioni o blocchi omogenei. La clausola **HAVING** applica poi un secondo livello di filtro, ma agisce in blocco sui gruppi interi, non sulle singole righe.
4. **SELECT (Target List):** Il sistema estrae finalmente le colonne o calcola le espressioni richieste dall'utente, applicando **DISTINCT** se richiesto per eliminare le combinazioni duplicate.
5. **ORDER BY:** A risultato ottenuto, si invoca l'algoritmo di Sort-Merge (visto nel capitolo dell'ottimizzazione) per restituire i dati al client secondo un ordinamento **ASC** o **DESC**.

2.2 Pattern Matching Avanzato: LIKE e SIMILAR TO

Quando il confronto non è un'esatta uguaglianza matematica (**=**), ma la ricerca di una sottostringa di testo (es. "trova i nomi che iniziano con A"), si usano operatori specifici.

- **Operatore LIKE:** È il pattern matching classico. Funziona in modalità *case-sensitive* (distingue maiuscole da minuscole) e si basa su due caratteri jolly (wildcard):

- `_` (Underscore): Impone il match posizionale con **esattamente un** carattere qualsiasi.
- `%` (Percentuale): Definisce una sequenza variabile da zero a infiniti caratteri qualsiasi. (Es. `LIKE 'A%a'` cattura "Anna" ma anche "Andrea").

Nota architetturale: PostgreSQL introduce la variante proprietaria `ILIKE`, che esegue lo stesso pattern ignorando il case-sensitivity (ottimo per ignorare gli errori di digitazione dell'utente finale).

- **Operatore SIMILAR TO:** Meno comune ma enormemente più potente, introduce la logica delle espressioni regolari (RegEx) POSIX nel motore SQL. Oltre ai jolly del `LIKE`, accetta operatori logici complessi direttamente nella stringa:
 - `[...]`: Definisce un range o un elenco di caratteri accettati. Es: `SIMILAR TO '[AFO]'` troverà parole che iniziano solo con A, F, oppure O.
 - `+` e `*`: Regolano la ripetitività dei match precedenti.

2.3 Funzioni di Aggregazione e l'Astrazione del GROUP BY

Invece di proiettare i dati puri in output, il sistema può comprimerli passando un intero insieme di righe attraverso una singola funzione matematica (`MAX`, `MIN`, `AVG`, `SUM`, `COUNT`).

L'utilizzo di un operatore aggregato **collassa inesorabilmente il risultato a una singola riga**. Se provassimo ad affiancare una colonna normale all'operatore (es. `SELECT Nome, MAX(Reddito)`), PostgreSQL solleverebbe un errore critico: il sistema non può restituire tutti i nomi di una tabella se deve condensare il reddito in una sola cella totale.

La soluzione strutturale a questo limite è il **GROUP BY**. Con questa clausola, istruiamo il motore del database a scorrere l'intero result-set (in RAM), individuare le tuple che presentano lo stesso esatto valore per gli attributi di raggruppamento e confinarle in una "scatola logica" invisibile. A questo punto, gli operatori di aggregazione agiranno **separatamente** all'interno di ciascuna scatola.

Ad esempio: `SELECT città, AVG(media) FROM Studente GROUP BY città;` calcolerà e restituirà un valore di media del tutto indipendente per ogni singola città rilevata.

L'Asimmetria tra WHERE e HAVING: Mentre il `WHERE` filtra chirurgicamente la materia prima (es. "scarta gli studenti fuoricorso"), l'`HAVING` è progettato per testare i risultati delle aggregazioni appena formatesi (es. "scarta le città che, a fine conteggio, hanno una media totale inferiore al 23"). Sotto il cofano, l'`HAVING` è applicabile *soltanto* a espressioni aggregate o agli attributi esplicitamente dichiarati nel `GROUP BY`.

2.4 L'Algebra Relazionale in Azione: Gli Operatori JOIN

La scomposizione di un database tramite il processo di normalizzazione produce tante tabelle leggere e performanti in scrittura, ma obbliga il programmatore a ricomporre il puzzle logico in lettura. L'operatore JOIN fonde lateralmente le tabelle.

- **INNER JOIN:** È la traduzione SQL pura del θ -Join (Theta-Join) algebrico. Date due tabelle, il database incrocia ogni riga di sinistra con ogni riga di destra e le restituisce al client **solo se** l'espressione booleana nella clausola ON risulta rigorosamente vera. Le righe orfane (quelle senza corrispondenze) di entrambe le tabelle vengono inesorabilmente annientate.
- **LEFT OUTER JOIN:** È l'ancora di salvezza per i dati orfani della tabella sinistra. L'operatore estrae tutti gli accoppiamenti validi ma, per ogni riga della prima tabella che non trova "compagni" sulla destra, inietta a forza una tupla riempiendo le colonne vuote destre con il valore NULL. Si tratta di un'operazione intrinsecamente **asimmetrica**: invertendo l'ordine dei nomi delle tabelle (TabellaA LEFT JOIN TabellaB vs TabellaB LEFT JOIN TabellaA), il motore processerà i dati e gestirà le perdite in modo radicalmente diverso.
- **RIGHT OUTER JOIN:** È speculare al precedente. Salva a tutti i costi le tuple della tabella scritta a destra, forzando i NULL nelle colonne vuote di sinistra.
- **FULL OUTER JOIN:** È un costrutto ibrido massivo. Si comporta come l'unione di un LEFT JOIN e un RIGHT JOIN simultanei, garantendo la totale assenza di perdita informativa da ambo le parti e innestando valori NULL dove le connessioni logiche risultano spezzate.

3 Interrogazioni Pratiche: Join e Logica del Database UNIVR

Passiamo dalla teoria astratta alla manipolazione reale dei dati. Quando ci troviamo di fronte a un database strutturato con centinaia di tabelle, come quello universitario, la difficoltà non risiede tanto nella conoscenza del singolo comando SQL, quanto nella capacità di "navigare" lo schema logico e unire le tabelle in modo sensato.

3.1 L'Architettura del Database e la Tabella Centrale

Per interrogare correttamente un database complesso, è fondamentale individuare le tabelle pivot (le tabelle ponte che risolvono le relazioni multi-a-molti). Nello schema UNIVR, il "cuore" pulsante è la tabella **InsErogato**.

Questa tabella non descrive solo un corso teorico, ma il corso **effettivamente tenuto** in un determinato anno accademico. Le sue colonne contengono regole di business vitali:

- **Gestione dei Moduli:** Un insegnamento può essere un blocco unico o diviso in moduli (es. Teoria + Laboratorio). La colonna `modulo` gestisce questa gerarchia. Se il valore è 0, la riga rappresenta l'insegnamento intero (Padre). Se il valore è maggiore di 0 (es. 1, 2), la riga rappresenta un singolo modulo figlio.
- **Le Repliche (Discriminanti):** Se un corso ha troppi studenti, viene diviso in sezioni parallele (es. "Sezione A", "Sezione B" o "Matricole Pari/Dispari"). In SQL, queste repliche sono tracciate tramite l'attributo `id_discriminante`.

L'errore più comune in questi laboratori è unire un professore direttamente alla materia, ignorando `InsErogato`. Il professore non insegna la "Programmazione" in senso astratto, ma insegna uno *specifico modulo* di Programmazione, di una *specifico sezione*, in uno *specifico anno*.

3.2 Interrogazioni Lineari e Condizionali

Una volta compreso lo schema, la stesura del codice segue un percorso logico prevedibile. Il costrutto `INNER JOIN` (o semplicemente `JOIN`) scarta inesorabilmente tutte le righe che non trovano un compagno nella tabella unita.

Quando viene richiesto di estrarre il nome degli insegnamenti offerti dal corso di "Informatica" nell'anno 2009/2010, la query deve partire dal corso di studi, scendere nella tabella erogazioni e infine risalire al catalogo degli insegnamenti, applicando i filtri finali:

```
SELECT DISTINCT I.nomeins
FROM CorsoStudi CS
JOIN InsErogato IE ON CS.id = IE.id_corsostudi
JOIN Insegn I ON IE.id_insegn = I.id
WHERE IE.annoaccademico = '2009/2010'
AND CS.nome = 'Laurea in Informatica';
```

Nota architetturale: L'uso del `DISTINCT` qui non è un vezzo stilistico, ma una necessità logica. Se il corso di Informatica ha diviso un insegnamento in 3 moduli (generando 3 righe in `InsErogato`), il `JOIN` puro stamperebbe 3 volte il nome della materia. Il `DISTINCT` ripulisce l'output.

3.3 Statistiche Complesse: GROUP BY e HAVING

Quando l'obiettivo non è stampare stringhe di testo, ma aggregare dati (contare docenti, sommare ore, calcolare medie), la clausola `GROUP BY` diventa il perno della query. Essa divide virtualmente la tabella in scompartimenti stagni in base alla colonna indicata (es. Raggruppa tutti i record per "Nome del Corso di

Laurea”). A quel punto, le funzioni COUNT() o SUM() agiranno non sull’intera tabella, ma su ogni singolo scompartimento.

Se si vuole calcolare quante ore totali insegna un professore, e tenere solo i professori che hanno più di un insegnamento, la query si struttura così:

```
SELECT P.Nome, P.Cognome,
COUNT(DISTINCT D.id_inserogato) AS NumIns,
SUM(D.orelez) AS OreTotali
FROM InsErogato IE
JOIN Docenza D ON IE.id = D.id_inserogato
JOIN Persona P ON P.id = D.id_persona
WHERE IE.annoaccademico = '2005/2006'
GROUP BY P.id, P.Nome, P.Cognome
HAVING COUNT(DISTINCT D.id_inserogato) > 1;
```

La differenza cronologica tra WHERE e HAVING è fondamentale. Il WHERE (anno = 2005) viene processato prima del raggruppamento e agisce sulle righe grezze, scartando tutte le lezioni tenute negli anni successivi. L’HAVING (> 1 insegnamento) viene processato *dopo* che i gruppi sono stati formati e conteggiati, scartando i professori che, nel 2005, hanno totalizzato un solo corso.

3.4 La Salvaguardia dei Dati: Uso Strategico del LEFT JOIN

Esistono interrogazioni in cui la logica distruttiva dell’INNER JOIN altera irrimediabilmente l’output richiesto. Se l’obiettivo è elencare tutti gli insegnamenti e mostrare a quale “quadrimestre” appartengono, ma **anche se** non ne hanno ancora uno assegnato, il JOIN standard cancellerebbe i corsi non schedulati.

La soluzione strutturale è il LEFT JOIN. Questo operatore garantisce la sopravvivenza incondizionata della tabella a sinistra della clausola, iniettando valori NULL nelle colonne mancanti a destra.

```
SELECT DISTINCT I.nomeins, PL.abbreviazione
FROM Insegn I
JOIN InsErogato IE ON I.id = IE.id_insegn
LEFT JOIN InsInPeriodo IP ON IE.id = IP.id_inserogato
LEFT JOIN PeriodoLez PL ON PL.id = IP.id_periodolez
WHERE IE.annoaccademico = '2006/2007'
AND IE.id_corsostudi = 4;
```

In questo scenario, se un corso non è ancora stato caricato nella tabella *InsInPeriodo*, la riga persiste e la colonna ‘abbreviazione’ (che conterrebbe il quadrimestre) stamperà a schermo il valore nullo, rispondendo perfettamente alla specifica funzionale.

4 Interrogazioni Complesse: Nidificazione, Insiemi e Viste

Quando la logica di estrazione dei dati supera i limiti di un singolo blocco `SELECT-FROM-WHERE`, il linguaggio SQL offre costrutti avanzati per frammentare il problema in sotto-query (nidificazione), combinare i risultati come se fossero insiemi matematici, o creare strutture logiche riutilizzabili chiamate viste.

4.1 L'Architettura delle Subquery

Una subquery non è altro che un'interrogazione SQL racchiusa tra parentesi tonde e annidata all'interno di una query principale (*Outer Query*). A seconda del punto in cui viene iniettata, la sua funzione architetturale cambia drasticamente:

- **Nidificazione nella clausola FROM:** In questo scenario, la subquery viene eseguita dal database *prima* di tutto il resto. Il risultato (un set di righe e colonne) viene trattato dal motore esattamente come se fosse una tabella fisica memorizzata sul disco, pronta per subire operazioni di JOIN o filtraggio da parte della query esterna.
- **Nidificazione nella clausola WHERE:** È lo scenario predominante. Qui la subquery agisce come un generatore dinamico di parametri per il filtro. Poiché il WHERE tradizionale è progettato per confrontare valori scalari singoli (es. `prezzo = 10`), l'introduzione di una subquery (che può restituire una lista di decine di valori) impone l'uso di operatori logici estesi per gestire il confronto tra uno scalare e un vettore.

4.2 Operatori Estesi per Subquery (ANY, ALL, IN)

Per raccordare il valore della riga corrente con il vettore restituito dalla subquery, SQL adotta i seguenti costrutti:

- **ANY / SOME:** La subquery deve restituire una singola colonna. Il database confronta il valore della riga in esame con ogni elemento del vettore interno. Se il test (es. `>` o `<`) risulta vero per **almeno uno** degli elementi, la riga sopravvive al filtro.
- **ALL:** Speculare al precedente, ma drasticamente più restrittivo. La riga sopravvive al filtro solo ed esclusivamente se il confronto risulta vero per **tutti** gli elementi del vettore interno. (Es. "Seleziona l'insegnamento se i suoi crediti sono `>` ALL (subquery che estrae i crediti di tutti gli altri corsi)").
- **IN / NOT IN:** Controlla la mera appartenenza di un elemento all'interno di un insieme. È sintatticamente equivalente a `= ANY`. La particolarità dell'IN è la possibilità di testare tuple multiple simultaneamente usando

il costrutto **ROW**, a patto che la subquery restituisca esattamente lo stesso numero di colonne.

4.3 L'Operatore EXISTS e il Data Binding

A differenza dei precedenti, l'operatore **EXISTS** disattiva l'estrazione dei dati dalla subquery e si limita a un controllo booleano di stato: restituisce **TRUE** se l'esecuzione della subquery produce almeno una riga, **FALSE** se produce un insieme vuoto.

Il vero potenziale di **EXISTS** si sprigiona con il meccanismo del **Data Binding** (o interrogazione correlata). In questa configurazione, la subquery interna non è autonoma, ma contiene un riferimento (es. **I.nome**) a una tabella dichiarata nel **FROM** della query esterna.

Sotto il cofano, questo innesca un pesante ciclo di esecuzione iterativa: il motore del database è costretto a "congelare" la subquery, leggere la prima riga della tabella esterna, iniettare il suo valore dentro la subquery, eseguirla, valutare l'**EXISTS**, e ripetere questo processo da zero per ogni singola riga della tabella principale.

4.4 Algebra degli Insiemi (UNION, INTERSECT, EXCEPT)

Se le subquery incastrano logicamente le condizioni, gli operatori insiemistici operano "in verticale", impilando blocchi di risultati separati per fonderli o sottrarli, simulando la teoria degli insiemi. Per far sì che l'operazione hardware non vada in crash, i due *result-set* in input devono essere **strutturalmente compatibili** (stesso numero di colonne e tipi di dato allineati).

- **UNION:** Esegue l'addizione. Accoda fisicamente le righe della seconda query in fondo a quelle della prima.
- **INTERSECT:** Esegue l'intersezione. Il motore confronta i due blocchi di memoria e restituisce unicamente le righe presenti simultaneamente in entrambi gli insiemi.
- **EXCEPT:** Esegue la sottrazione (differenza). Rimuove spietatamente dal primo result-set qualsiasi riga che trovi una corrispondenza esatta nel secondo result-set.

Nota architetturale: Tutti gli operatori insiemistici, per impostazione predefinita di SQL, scatenano un algoritmo di Sort-Merge occulto per individuare e annientare i duplicati. Per disattivare questo costoso filtraggio e preservare le ridondanze, è necessario affiancare la keyword **ALL** (es. **UNION ALL**).

4.5 Le Viste: Astrazione e Tabelle Virtuali

Una **Vista** (**VIEW**) è un costrutto del DDL che permette di associare un'etichetta testuale persistente a una query complessa.

Sotto il cofano, quando si crea una vista, PostgreSQL **non muove un singolo byte di dati** e non alloca spazio su disco per i risultati. Registra semplicemente la stringa SQL all'interno del dizionario di sistema. Ogni volta che un utente inserisce il nome della vista in una clausola **FROM**, l'ottimizzatore espande macchinalmente il nome, sostituisce la macro con il codice originale e ricalcola l'intero albero di esecuzione "al volo". Questo garantisce che la vista mostri sempre i dati freschi e aggiornati all'ultimo istante.

PostgreSQL supporta anche le **TEMP VIEW** (Viste Temporanee), un'estensione architetturale che alloca la definizione logica solo nella memoria della sessione corrente, distruggendola automaticamente non appena il client chiude la connessione con il postmaster.

5 Ottimizzazione delle Prestazioni e Piani di Esecuzione (EXPLAIN)

In ambienti di produzione reali, dove le tabelle ospitano milioni di righe, una query formalmente corretta può rivelarsi ingegneristicamente disastrosa. Quando si eseguono **JOIN** multipli, il modo in cui il DBMS esplora i dati fa la differenza tra una risposta in 2 millisecondi e un blocco del server. Lo strumento principale per svelare l'architettura decisionale del database e migliorare le prestazioni è la gestione degli Indici e l'analisi dei Piani di Esecuzione.

5.1 Indici: Costi, Benefici e Implementazione

Un indice è una struttura dati ausiliaria parallela alla tabella. Il suo scopo esclusivo è abbattere i tempi di ricerca. L'implementazione di un indice, tuttavia, impone un **dazio strutturale**: per ogni operazione di **INSERT**, **UPDATE** o **DELETE** effettuata sulla tabella principale, il DBMS è obbligato a riorganizzare e riscrivere anche l'albero dell'indice, zavorrando le prestazioni di scrittura.

La regola ingegneristica ferrea prescrive di definire gli indici chirurgicamente, basandosi esclusivamente sui pattern di lettura (le query) statisticamente più frequenti.

5.1.1 Sintassi e Comandi Interni

Mentre le **PRIMARY KEY** e i vincoli **UNIQUE** generano indici automaticamente, per ogni altro attributo è necessaria un'istruzione esplicita:

```
CREATE INDEX nome_indice ON NomeTabella (nome_attributo);
```

Per forzare il motore a ricalcolare le metriche probabilistiche di distribuzione dei dati, vitali affinché l'ottimizzatore scelga di usare effettivamente il nuovo indice, si deve lanciare:

```
ANALYZE NomeTabella;
```

5.1.2 Il Problema della Localizzazione e l'uso di `varchar_pattern_ops`

La creazione di un B-Tree standard (`USING btree`) non è sempre sufficiente per le stringhe. Quando il server PostgreSQL è configurato con un "Locale" specifico (es. la lingua italiana `it_IT.UTF-8` invece dello standard C), le regole di collazione (come vengono ordinati alfabeticamente i caratteri accentati o minuscoli/maiuscoli) si alterano profondamente.

Sotto il cofano, questa localizzazione "acceca" il B-Tree standard di fronte agli operatori `LIKE` o di disuguaglianza (`<`, `>`). Per forzare il B-Tree a ignorare la grammatica locale e riallinearsi ai byte puri per far funzionare correttamente il `LIKE 'Prefisso%'`, è necessario imporre un flag specifico durante la creazione:

```
CREATE INDEX nome_idx ON Tabella (attributo varchar_pattern_ops);
```

5.1.3 Indici Multi-Attributo e Indici di Espressione

- **Multi-Attributo:** Se un'applicazione filtra regolarmente usando un `AND` tra due specifiche colonne, un singolo indice che racchiude entrambe le colonne è computazionalmente superiore a due indici separati. Tuttavia, se la query usa un `OR`, la matematica dell'albero si rompe, costringendo il database ad abbandonare l'indice multi-attributo e scansionare brutalmente l'intera tabella.
- **Indici di Espressione:** Se le interrogazioni manipolano costantemente il dato in lettura (ad esempio `LOWER(nome_colonna)`), l'indice sulla colonna originale diviene inservibile. È possibile creare l'indice non sul dato grezzo, ma direttamente sul risultato della funzione matematica: `CREATE INDEX idx_low ON Tab (LOWER(attr) varchar_pattern_ops);`.

5.2 Decodificare l'Architettura: Il comando `EXPLAIN`

Il comando `EXPLAIN` (seguito dalla query) obbliga l'ottimizzatore a svelare a schermo l'albero decisionale completo (Piano di Esecuzione) senza muovere o alterare i dati sul disco. Aggiungendo `ANALYZE`, il comando non si limita a teorizzare, ma esegue realmente la query (cronometrando con precisione ogni operazione hardware) e restituisce un delta tra la teoria e la pratica.

5.2.1 Nodi Architettureali nel Piano di Esecuzione

La lettura di un `EXPLAIN` si effettua partendo dal basso verso l'alto (dalle foglie dell'albero alla radice):

- **Seq Scan (Sequential Scan):** Il nemico delle prestazioni su grandi tabelle. Indica che il sistema, in assenza di indici utili o a causa di regole fallaci, sta leggendo spietatamente la tabella blocco per blocco dall'inizio alla fine.
- **Index Scan:** Ottimale. Il motore percorre l'indice, recupera il puntatore ed estrae il dato esatto.

- **Bitmap Index Scan e Bitmap Heap Scan:** L'ottimizzatore divide il lavoro. Il *Bitmap Index Scan* (il figlio) usa l'indice esclusivamente per mappare in un vettore compatto gli indirizzi di memoria delle righe pertinenti. Subito dopo, passa il vettore al nodo padre *Bitmap Heap Scan* che si reca sul disco, recupera le righe in blocco e applica la *Recheck Cond* per filtrare eventuali falsi positivi strutturali.
- **I Nodi di JOIN:** L'output svela anche come il motore ha deciso di legare fisicamente le tabelle:
 - **Nested Loop:** Un ciclo "for" software annidato ($R \times S$), mortale se non mitigato dagli indici.
 - **Hash Join:** Costruisce in RAM una tabella hash compatta per la relazione più piccola e spazzola la seconda verificando gli agganci logici.
 - **Merge Join:** Ordina preventivamente i due blocchi di dati (*Sort Key*) prima di fonderli parallelamente a scorrimento rapido.

5.2.2 I Numeri dell'Output

Osservando la dicitura (`cost=18.40..134.62 rows=15 width=63`), le metriche chiave si traducono in:

- **cost (primo valore):** Il costo logico stimato per elaborare ed estrarre fisicamente la *prima riga* utile (il setup).
- **cost (secondo valore):** Il peso computazionale stimato per completare *l'intera* operazione di quel nodo. L'obiettivo supremo dell'ottimizzatore è abbattere questo numero.
- **rows:** Le righe che il sistema stima di restituire. Con `EXPLAIN ANALYZE`, il comando stamperà di fianco a questo numero il parametro `actual time` (i veri millisecondi spesi) e `actual rows` (le righe fisicamente recuperate dal disco).
- **loops:** Se un'operazione fa parte di un ciclo (es. un Nested Loop interno), questo valore indica quante volte il nodo ha dovuto ripetere la medesima azione hardware, smascherando i colli di bottiglia causati dalla mancanza di indici.

6 Controllo di Concorrenza: L'Architettura MVCC in PostgreSQL

Quando si abbandona la teoria dei grafi dei conflitti e si analizza un DBMS di produzione come PostgreSQL, si scopre che il rigido protocollo 2PL (Two-Phase Locking) classico, pur essendo matematicamente inattaccabile, è troppo

penalizzante per le prestazioni di un sistema multi-utente. Se ogni lettura bloccasse le scritture (e viceversa), il parallelismo crollerebbe. Per risolvere questo limite hardware-software, PostgreSQL abbandona i lock in lettura e adotta un'architettura radicalmente diversa: l'**MVCC (Multiversion Concurrency Control)**.

6.1 Il Cuore dell'MVCC: Niente Lock sulle Letture

Il principio fondamentale dell'MVCC è che **i lettori non bloccano mai gli scrittori, e gli scrittori non bloccano mai i lettori**. Sotto il cofano, PostgreSQL ottiene questo risultato non sovrascrivendo fisicamente i dati sul disco quando un utente fa un **UPDATE**. Invece, crea una **nuova versione** della tupla e la affianca a quella vecchia, marcandole con degli ID di transazione nascosti (**xmin** per chi l'ha creata, **xmax** per chi l'ha "cancellata/modificata").

Quando una transazione inizia, il DBMS le fornisce uno *Snapshot* (un'istantanea logica temporale) del database. La transazione vedrà sempre e solo le versioni delle tuple che erano già consolidate (**COMMITTED**) nell'istante esatto in cui ha scattato la foto, ignorando totalmente le nuove versioni in corso di scrittura da parte di altri utenti.

L'unico momento in cui il sistema è costretto a usare i lock (sospendendo le operazioni) è quando due transazioni tentano un **conflitto Scrittura-Scrittura**. Se T_1 sta modificando la tupla X , e T_2 tenta di modificare o cancellare la stessa tupla X , T_2 viene messa fisicamente in coda di attesa finché T_1 non esegue il **COMMIT** o il **ROLLBACK**.

6.2 I Livelli di Isolamento: Dalla Tolleranza alla Paranoia

Sebbene l'MVCC protegga intrinsecamente i lettori, il grado di isolamento temporale della fotografia dei dati è configurabile via software tramite il comando **SET TRANSACTION ISOLATION LEVEL**. Aumentare l'isolamento riduce le anomalie, ma aumenta il tasso di fallimento delle transazioni concorrenti.

6.2.1 1. Read Committed (Il Default)

È il livello base in PostgreSQL (che ingloba anche il *Read Uncommitted*, non supportato fisicamente per via della natura stessa dell'MVCC).

- **Come funziona:** L'istantanea non è fissa per tutta la transazione, ma viene **rinnovata all'inizio di ogni singolo comando SQL** (**SELECT**, **UPDATE**, **DELETE**) all'interno del blocco.
- **Anomalie tollerate:** Soffre di *Lettura Inconsistente*. Se T_1 esegue due volte la stessa **SELECT**, e nel mezzo T_2 modifica un dato e fa commit, la seconda **SELECT** di T_1 scatterà una nuova foto e vedrà il dato cambiato. Soffre inoltre di *Aggiornamento Fantasma* (valutazioni logiche sballate su aggregati) e *Inserimento Fantasma*.

6.2.2 2. Repeatable Read (La Foto Bloccata)

- **Come funziona:** L'istantanea del database viene **scattata e bloccata al primissimo comando della transazione**. Tutte le `SELECT` successive vedranno esattamente gli stessi identici dati dell'istante iniziale, ignorando categoricamente i `COMMIT` effettuati nel frattempo dalle altre sessioni.
- **Gestione dei conflitti in scrittura:** Se T_1 (in Repeatable Read) tenta un `UPDATE` su una riga che è stata appena modificata e "committata" da T_2 , PostgreSQL **non la mette in coda**. Al contrario, per preservare l'integrità logica della foto iniziale, fa schiantare la transazione T_1 lanciando un errore fatale (`ERROR: could not serialize access due to concurrent update`).
- **Anomalie tollerate:** Evita quasi tutto, ma non la *Mancata Serializzazione*. Se T_1 e T_2 leggono contemporaneamente un valore massimo per calcolare un nuovo ID e fanno un `INSERT`, genereranno due righe con lo stesso ID senza ostacolarsi, rompendo la logica sequenziale.

6.2.3 3. Serializable (L'Esecuzione Strict)

- **Come funziona:** È il livello più paranoico. Il database analizza le dipendenze logiche tra letture e scritture (Read/Write dependencies). Se rileva che l'intreccio parallelo di due transazioni genererebbe un risultato finale diverso da quello che si otterrebbe mettendole rigorosamente in fila indiana (una dopo l'altra), interviene brutalmente.
- **Gestione dei conflitti:** Appena individua il rischio di anomalia (come il doppio `INSERT` calcolato sull'esempio precedente), abortisce immediatamente una delle transazioni per preservare la coerenza assoluta, restituendo un errore specifico.

6.3 Architettura Software: La Gestione Applicativa dei Fallimenti

Da un punto di vista ingegneristico, l'adozione dei livelli `Repeatable Read` o `Serializable` ribalta la responsabilità dal database al programmatore.

Poiché il motore abortirà regolarmente le transazioni per proteggere i dati in caso di collisioni concorrenti, sollevando un'eccezione di sistema codificata come `SQLSTATE = '40001'`, l'applicazione esterna (in Python, Java, o C++) deve essere progettata per **intercettare questo specifico errore**. Quando il backend rileva il codice 40001, non deve far crashare il sistema utente, ma deve implementare un blocco logico di "Retry", ordinando al client di rimettere in esecuzione l'intero blocco `BEGIN...COMMIT` da zero.

Questo approccio ottimistico (provo, se fallisco riprovo) è computazionalmente e architetturealmente molto più performante rispetto all'uso dei lock espliciti pessimistici (come `SELECT FOR UPDATE`), che "congelerebbero" fisicamente le tabelle costringendo i thread in code hardware infinite.

7 Sviluppo Applicativo: Python e Integrazione con PostgreSQL (DB-API 2.0)

La potenza di un DBMS relazionale rimane inespressa se non interfacciata con un linguaggio di programmazione di alto livello in grado di gestire la logica applicativa, le interfacce utente e l'I/O complesso. In questo modulo analizziamo l'integrazione tra Python e PostgreSQL.

7.1 Python: Paradigma e Strutture Dati Native

A differenza di linguaggi staticamente tipizzati e compilati come Java o C, Python è un linguaggio **interpretato** a **tipizzazione dinamica**. Il tipo di una variabile non viene dichiarato a monte, ma viene inferito a runtime dal motore di esecuzione in base al valore assegnato, e può mutare durante l'esecuzione (es. `x = 2` e poi `x = 'Due'`).

Dal punto di vista della formattazione, Python abbandona le parentesi graffe per delimitare i blocchi di codice, affidandosi interamente all'**indentazione** rigorosa tramite spazi o tabulazioni.

L'architettura del linguaggio offre primitive di dati estremamente potenti senza richiedere l'importazione di librerie esterne (come avviene con `java.util.*`):

- **Liste (List):** Array dinamici e mutabili (`[1, 'a', 2]`).
- **Tuple:** Versioni blindate e **immutabili** delle liste (`(1, 'a', 2)`). Un tentativo di riassegnazione (es. `t[0] = 5`) causerà un crash immediato del programma.
- **Set:** Collezioni matematiche pure. Inserendo elementi duplicati in un Set (`{1, 2, 2}`), il motore li ignorerà scartandoli automaticamente, mantenendo l'unicità.
- **Dizionari (Dict):** L'equivalente concettuale delle *HashMap*. Mappano chiavi univoche a valori (`{'nome': 'Bob', 'eta': 30}`).

7.2 L'Astrazione DB-API 2.0 e la libreria psycopg2

Python non parla nativamente il dialetto binario di PostgreSQL. Per stabilire un ponte comunicativo, lo standard ufficiale Python definisce un protocollo astratto chiamato **DB-API v2.0**. La libreria `psycopg2` è l'implementazione pratica di questo protocollo, scritta quasi interamente in linguaggio C per garantire prestazioni massime interfacciandosi con la libreria nativa `libpq` di PostgreSQL.

Il flusso architetturale per comunicare con il database segue uno schema rigido:

1. **Connessione:** Si apre il tunnel TCP/IP verso il server usando le credenziali (`psycopg2.connect()`).

2. **Cursore:** Si genera un oggetto Cursore (`conn.cursor()`). Il cursore è il "braccio armato" che invia materialmente le query SQL (`execute()`) e preleva i risultati dal buffer (`fetchone()`, `fetchmany()`, o iterazione).
3. **Gestione Transazionale:** Di default, la libreria apre silenziosamente una transazione non appena si invia il primo comando. È **obbligatorio** invocare `conn.commit()` per consolidare le modifiche su disco. In caso contrario, alla chiusura del programma, PostgreSQL eseguirà un `ROLLBACK` spietato di tutte le operazioni rimaste in sospeso.
4. **Chiusura:** I cursori e la connessione vanno chiusi per liberare la memoria RAM sul server. L'uso dei blocchi di *context management* in Python (`with conn:`) automatizza elegantemente il `commit` e la chiusura delle risorse.

7.3 Sicurezza e Sanitizzazione: SQL Injection

La vulnerabilità applicativa più letale nello sviluppo di database è la **SQL Injection**. Si verifica quando un programmatore inesperto costruisce la stringa SQL concatenando brutalmente input grezzi forniti dall'utente:

```
cur.execute("SELECT * FROM users WHERE name = '" + utente_input + "'")
```

Se l'utente inserisce come input la stringa letale `a'; DROP TABLE users CASCADE;` --, il motore Python la concatenerà fedelmente e invierà al server PostgreSQL l'ordine esplicito di annientare la tabella, distruggendo il database.

Per disinnescare alla radice questa vulnerabilità, `psycopg2` impone l'uso del **Parameter Binding** (passaggio sicuro dei parametri) tramite il segnaposto `%s`.

```
cur.execute("SELECT * FROM users WHERE name = %s", (utente_input,))
```

Sotto il cofano, la libreria intercetta il valore della variabile, esegue l'escaping dei caratteri pericolosi (come apici o punti e virgola) e forza il motore SQL a interpretare l'input **esclusivamente come una stringa di testo puro**, neutralizzando qualsiasi tentativo di iniezione di comandi eseguibili. Inoltre, la libreria converte automaticamente i tipi Python nativi nei corrispondenti domini SQL (es. trasforma l'oggetto `datetime` di Python in un `TIMESTAMP` per PostgreSQL, l'oggetto `Decimal` in `NUMERIC`, e l'oggetto `None` in `NULL`).

8 MongoDB: Modello a Documenti e Interrogazioni

Mentre i database relazionali costringono i dati all'interno di griglie bidimensionali rigide, MongoDB adotta un'architettura *schema-less* orientata ai documenti. Sotto il cofano, le informazioni non vengono salvate come record in blocchi tabellari, ma serializzate sul disco in formato BSON (Binary JSON). Questo cambiamento strutturale altera radicalmente la nomenclatura e l'interazione con il motore: le "tabelle" diventano **Collezioni** e le "righe" diventano **Documenti**. L'interazione non avviene tramite linguaggio SQL, ma attraverso comandi JavaScript impartiti via terminale (`mongosh`) o driver applicativi.

8.1 Operazioni CRUD: L'Astrazione del Formato JSON

L'inserimento e la manipolazione dei dati in MongoDB avvengono passando direttamente oggetti JSON al motore del database.

Creazione (Insert): Quando si invoca un comando come `insertOne()` o `insertMany()`, il database esegue un'operazione fondamentale a basso livello: se il programmatore non lo ha fornito esplicitamente, il sistema genera e inietta automaticamente un attributo `_id` di tipo **ObjectID**. Questa chiave primaria generata via hardware, lunga esattamente 12 byte, contiene un timestamp, un identificatore della macchina e un contatore, garantendo l'unicità assoluta del documento nel cluster distribuito senza richiedere un generatore di sequenze centralizzato e bloccante.

Lettura e Filtraggio (Find): L'estrazione dei dati avviene tramite il metodo `find()`. Al posto della classica clausola `WHERE`, l'ottimizzatore di MongoDB riceve in pasto un documento JSON contenente i criteri di filtro. Poiché la sintassi strutturale non ammette operatori matematici standard come `<` o `>`, il motore esegue il parsing di chiavi speciali precedute dal simbolo del dollaro (`$`). Ad esempio, per trovare prezzi inferiori a 30, si passa un oggetto interno `{lt: 30}`. Il motore supporta nativamente logiche di pattern matching tramite espressioni regolari (`{regex: "^V"}`) e operatori logici complessi come `{or: [...]}` e `{in: [...]}`. L'ottimizzatore analizza questo JSON per decidere se scansionare l'intera collezione blocco per blocco (*Collection Scan*) o se scendere lungo un albero B-Tree per un *Index Scan*.

Modifica (Update vs Replace): A livello di cicli di lettura/scrittura su disco (I/O), esiste una profonda differenza ingegneristica tra aggiornare un campo e rimpiazzare un documento. Il comando `updateOne()` richiede l'uso dell'operatore di modifica `{set: ...}`; questo ordina al motore di sovrascrivere fisicamente *solo* i byte di quel campo specifico, lasciando intatto il resto della struttura BSON. Al contrario, l'istruzione `replaceOne()` ordina al sistema di annientare completamente la struttura del documento originale (preservando unicamente l'ObjectID intatto) e riscriverla da zero con il nuovo payload JSON fornito in input.

8.2 Architettura del Dato: Incapsulamento vs Riferimenti

Il problema prestazionale più grande nella progettazione in MongoDB è la modellazione delle relazioni, non esistendo chiavi esterne native supportate da vincoli `CASCADE` o `RESTRICT` a livello di motore. L'ingegnere dei dati deve bilanciare memoria e velocità scegliendo tra due paradigmi fisici:

- **Incapsulamento (Embedding o Nidificazione):** Si inserisce il documento figlio direttamente all'interno del documento padre (ad esempio, annidando un intero array di recensioni dentro il documento del singolo museo). Dal punto di vista delle prestazioni in lettura, è una soluzione insuperabile: il disco compie un'unica operazione di I/O sequenziale estraendo l'intero aggregato in un colpo solo, eliminando alla radice la necessità di calcolare costosi JOIN. Tuttavia, questo approccio fa esplodere la

ridondanza dei dati (consumando rapidamente RAM e storage) e si scontra contro l'*hard-limit* inossidabile di 16 Megabyte imposto dal motore per la dimensione massima di un singolo documento BSON.

- **Riferimenti (Referencing):** È l'approccio relazionale puro. Si salva unicamente l'`ObjectID` del documento padre all'interno del documento figlio. L'occupazione di memoria RAM viene brutalmente ottimizzata e la ridondanza azzerata, ma la penalità si sconta interamente in fase di lettura: per recuperare l'informazione completa, il database o il software applicativo sono costretti a eseguire accessi multipli sparsi sul disco o generare chiamate di rete sequenziali aggiuntive.

8.3 La Pipeline di Aggregazione e i JOIN Logici

Quando l'interrogazione richiede calcoli statistici, raggruppamenti su decine di migliaia di righe o l'incrocio di collezioni separate, il metodo di base `find()` risulta inefficace. MongoDB sposta quindi l'elaborazione su un paradigma di *stream-processing* chiamato **Pipeline di Aggregazione** (`aggregate()`).

L'elaborazione dei byte avviene a stadi strettamente sequenziali, come una catena di montaggio all'interno della memoria RAM: i dati entrano, subiscono una prima trasformazione hardware e il *result-set* rimpicciolito o alterato viene passato allo stadio successivo. I principali stadi riproducono l'algebra relazionale:

- **\$match:** L'equivalente del `WHERE`. Deve essere sempre posto all'inizio della pipeline per eseguire il filtraggio grezzo e scartare i documenti inutili, riducendo immediatamente il volume dei byte in transito verso le fasi più pesanti.
- **\$group:** L'equivalente del `GROUP BY`. Accumula i dati in memoria centrale basandosi su una chiave di raggruppamento (es. `_id: "$citta"`) e calcola parallelamente metriche statistiche tramite operatori di accumulazione hardware come `$avg`, `$sum` o `$max`.
- **\$sort:** Invoca l'algoritmo di ordinamento rapido in RAM sui risultati.

Nonostante l'assenza di rigidità relazionale, lo stadio `$lookup` fornisce a MongoDB la capacità di fondere documenti appartenenti a collezioni diverse, simulando fisicamente il comportamento di una **LEFT OUTER JOIN**. L'ottimizzatore analizza il campo locale del documento in transito e cerca corrispondenze indicizzate nella collezione esterna target. Tutti i documenti trovati non vengono fusi in colonne piatte, ma vengono impacchettati dinamicamente in un nuovo *Array* JSON e iniettati al volo all'interno del documento risultante, mantenendo logicamente in vita anche i documenti per cui il match ha fallito.